

SRS For Conversational Chatbot using Seq2Seq Models

Table of Contents

1. Introduction
 - 1.1. Purpose
 - 1.2. Document Conventions
 - 1.3. Intended Audience
 - 1.4. Project Scope
 - 1.5. References
2. Overall Description
 - 2.1. Product Perspective
 - 2.2. Product Functions
 - 2.3. User Classes and Characteristics
 - 2.4. Operating Environment
 - 2.5. Design and Implementation Constraints
 - 2.6. Assumptions and Dependencies
3. System Features (Functional Requirements)
 - 3.1. Feature 1: Data Preprocessing
 - 3.2. Feature 2: Seq2Seq Model Implementation
 - 3.3. Feature 3: Model Training
 - 3.4. Feature 4: Response Generation
4. Non-Functional Requirements
 - 4.1. Performance Requirements
 - 4.2. Software Quality Attributes
5. **Appendix A: Glossary**

1. Introduction

1.1. Purpose

In today's digital era, there is a high demand for intelligent conversational agents in areas like customer service, personal assistance, and education. Traditional chatbots often rely on rigid, fixed rules or simple keyword matching, which limits their ability to handle the nuances and unpredictability of natural human conversation.

The purpose of this project is to design, build, and train a conversational chatbot that overcomes these limitations by leveraging a Sequence-to-Sequence (Seq2Seq) neural network model. This learning-based approach will enable the chatbot to understand user input in natural language and generate simple, contextually correct replies from a Q&A corpus, thereby improving the overall quality of user interaction.

1.2. Document Conventions

This document follows a standard SRS format. All requirements are assigned a unique identifier (e.g., REQ-XXX) for traceability.

1.3. Intended Audience

This document is intended for project developers, AI/ML engineers, and project supervisors. It provides a detailed blueprint for the system's functionality and serves as the primary guide for development and testing.

1.4. Project Scope

The scope of this project is to develop a proof-of-concept chatbot that demonstrates the viability of the Seq2Seq architecture for conversational AI. The primary deliverable is a working chatbot model that can generate simple, relevant phrases in response to input queries.

In-Scope Activities:

- Preprocessing of a conversational dataset (e.g., Cornell Movie Dialog Corpus), including tokenization, vocabulary building, and creating embeddings.
- Implementation of a Sequence-to-Sequence (Seq2Seq) model using an RNN, LSTM, or GRU architecture.
- Training the model on prepared question-and-answer pairs.
- Testing the chatbot's ability to generate simple conversational replies to novel inputs.
- Implementation of Beam Search for improved response decoding.

Out-of-Scope (Future Work):

- Handling complex, multi-turn dialogues and maintaining long-term context.
- Implementing more advanced models, such as Transformers (BERT, GPT).
- Deployment of the chatbot on a real-world platform (e.g., web interface, mobile application, or messaging app integration).

1.5. References

- Cornell Movie Dialog Corpus (or an alternative custom Q&A dataset).
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*.

2. Overall Description

2.1. Product Perspective

This project is a new, self-contained system. It serves as a foundational prototype and a research tool to explore the application of Seq2Seq models in conversational AI. It is not a component of a larger system at this stage but is designed to be extensible.

2.2. Product Functions

The major functions of the system are:

1. **Data Ingestion and Preprocessing:** The system will read a raw text corpus, clean it, tokenize it, and transform it into a numerical format suitable for training a neural network.
2. **Model Training:** The system will implement and train an encoder-decoder (Seq2Seq) model to learn mappings from input sequences (queries) to output sequences (responses).
3. **Response Generation:** The system will use the trained model to take a user's text query and generate a contextually relevant text response.

2.3. User Classes and Characteristics

- **Developer/Researcher:** The primary user of this system. They possess technical expertise in machine learning and Python. They will interact with the system via code to preprocess data, train the model, and evaluate its performance.
- **End User (for demonstration):** A non-technical user who will interact with the trained chatbot through a simple command-line interface to test its conversational abilities.

2.4. Operating Environment

- **Platform:** The system will be developed and run in a Python 3.x environment.
- **Libraries:** It will depend on standard data science and deep learning libraries, including but not limited to: TensorFlow or PyTorch, Scikit-learn, NLTK, and Pandas.
- **Hardware:** A machine with a compatible GPU is highly recommended for efficient model

training.

2.5. Design and Implementation Constraints

- The core architecture must be a Sequence-to-Sequence (Seq2Seq) model.
- The recurrent units within the encoder and decoder must be one of the following: standard RNN, LSTM, or GRU.

2.6. Assumptions and Dependencies

- **Assumption:** The chosen dataset is of sufficient size and quality to train a model capable of generating coherent, simple responses.
- **Dependency:** The project is dependent on the availability and stable versions of the required open-source Python libraries (TensorFlow/PyTorch, etc.).

3. System Features (Functional Requirements)

3.1. Feature 1: Data Preprocessing

- **Description:** The system shall process a raw text corpus to prepare it for model training.
- **Requirements:**
 - **REQ-001:** The system shall parse the input corpus into question-answer pairs.
 - **REQ-002:** The system shall perform text tokenization on all sentences.
 - **REQ-003:** The system shall build a vocabulary of unique tokens and map each token to a unique integer ID.
 - **REQ-004:** The system shall convert all text sequences into integer sequences based on the vocabulary.
 - **REQ-005:** The system shall pad all sequences to a fixed length for batch processing.

3.2. Feature 2: Seq2Seq Model Implementation

- **Description:** The system shall contain a complete implementation of the Seq2Seq architecture.
- **Requirements:**
 - **REQ-006:** The system shall implement an Encoder module that processes the input sequence and outputs a context vector (thought vector).
 - **REQ-007:** The system shall implement a Decoder module that takes the context vector and generates an output sequence token by token.
 - **REQ-008:** The model shall use word embedding to convert integer tokens into dense vectors.

3.3. Feature 3: Model Training

- **Description:** The system shall be capable of training the implemented Seq2Seq model.
- **Requirements:**
 - **REQ-009:** The system shall load the preprocessed data in batches.
 - **REQ-010:** The system shall train the model by minimizing a suitable loss function

(e.g., sparse categorical cross-entropy).

- **REQ-011:** The system shall provide a mechanism to save the trained model weights and vocabulary for later use.

3.4. Feature 4: Response Generation

- **Description:** The system shall use the trained model to generate responses to user queries.
- **Requirements:**
 - **REQ-012:** The system shall provide an interface (e.g., command line) to accept a string of text as user input.
 - **REQ-013:** The input text shall be preprocessed using the same steps as the training data.
 - **REQ-014:** The system shall feed the processed input into the trained model to generate an output sequence of token IDs.
 - **REQ-015:** The system shall implement a Beam Search decoding strategy to generate the final response.
 - **REQ-016:** The system shall convert the output token IDs back into a human-readable string of text.

4. Non-Functional Requirements

4.1. Performance Requirements

- **PERF-01:** The response generation (inference) for a typical query (under 20 words) should complete in under 5 seconds on the target hardware.
- **PERF-02:** The model training process should be able to run without memory errors on a system with at least 16GB RAM and an 8GB VRAM GPU.

4.2. Software Quality Attributes

- **QUAL-01 (Maintainability):** The code shall be well-documented and organized into modular components (e.g., data preprocessing, model definition, training loop) to facilitate understanding and future modifications.
- **QUAL-02 (Reusability):** The data preprocessing pipeline should be designed to be reusable with other compatible datasets.

Appendix A: Glossary

- **Seq2Seq (Sequence-to-Sequence):** A neural network architecture that transforms an input sequence into an output sequence. It consists of an Encoder and a Decoder.
- **Encoder:** The part of the Seq2Seq model that processes the input sequence and compresses its information into a fixed-size context vector.
- **Decoder:** The part of the Seq2Seq model that takes the context vector from the Encoder and generates the output sequence one item at a time.
- **Tokenization:** The process of breaking down a stream of text into smaller units called tokens (e.g., words or sub-words).
- **Embeddings (Word Embeddings):** Dense vector representations of words, where words with similar meanings have similar vector representations.
- **Beam Search:** A decoding algorithm used in sequence generation that explores multiple potential output sequences simultaneously to find a more optimal result than a greedy search.